

Radix Sort (sorting algorithm)

Part 1: Radix Sort

Why do we need Radix Sort?

Normally, sorting algorithms compare two elements.

For example:

5 < 9 ?

8 > 2 ?

Every comparison sort (Quick Sort, Merge Sort, Heap Sort) has a lower bound

$O(n \log n)$

$O(n \log n)$

because comparisons cannot do better.

But integers have digits, so instead of comparing numbers, we sort by each digit.

This avoids comparisons.

Example

Suppose

170
45
75
90
802
24
2
66

We sort one digit at a time.

Starting from

Units digit



Tens digit



Hundreds digit

This is called

LSD Radix Sort
(Least Significant Digit First)

Why start from the rightmost digit?

Imagine

329
328
327

If we first sorted by hundreds,

329
328
327

Nothing changes because all hundreds are 3.

But if we first sort by ones

327
328
329

then later sorting by tens and hundreds (using a stable sort) preserves that order.

This is why LSD works.

The Most Important Requirement

Every digit sort must be stable.

Stable means

If two numbers have the same current digit,
their previous order must remain unchanged.

Example

21
22
23

Sorting by tens digit

All have

2

So the order must stay

21
22
23

Not

22
21
23

Otherwise Radix Sort fails.

Which sorting algorithm is stable?

We use Counting Sort.

So Radix Sort is actually

Repeat

Counting Sort

for every digit.

Step-by-Step Example

Let's sort

- 170
- 45
- 75
- 90
- 802
- 24
- 2
- 66

Pass 1

Sort according to

Units digit

Number	Units
170	0
45	5
75	5
90	0
802	2
24	4
2	2
66	6

Arrange according to units

170
90
802
2
24
45
75
66

Notice

45

75

Both have

5

Their order stays

45 before 75

because Counting Sort is stable.

Pass 2

Now sort according to

Tens digit

Current array

170
90
802
2
24
45
75
66

Tens digits

170 ->7

90 ->9

802 ->0

2 ->0

24 ->2

45 ->4

75 ->7

66 ->6

Stable sort gives

802

2

24

45

66

170

75

90

Notice

170

75

Both have tens digit

7

Previous order remains.

Pass 3

Sort according to

Hundreds

Digits

```
802 ->8
170 ->1
others ->0
```

Result

```
2
24
45
66
75
90
170
802
```

Sorted!

Visual Summary

Original

```
170
45
75
90
802
24
2
66
```



Units

170
90
802
2
24
45
75
66



Tens

802
2
24
45
66
170
75
90



Hundreds

2
24
45
66
75
90
170
802

Time Complexity

Suppose

Maximum digits = d

Each Counting Sort

$O(n)$

Total

$d \times O(n)$

=

$O(dn)$

For 32-bit integers,

$d = 10$

(constant)

So

$O(n)$

```

void countingSort(vector<int>& nums, int exp) {
    int n = nums.size();

    vector<int> output(n);

    vector<int> cnt(10,0);

    for(int x : nums)
        cnt[(x/exp)%10]++;

    for(int i=1;i<10;i++)
        cnt[i]+=cnt[i-1];

    for(int i=n-1;i>=0;i--){

        int digit=(nums[i]/exp)%10;

        output[cnt[digit]-1]=nums[i];

        cnt[digit]--;
    }

    nums=output;
}

void radixSort(vector<int>& nums){

    int mx=*max_element(nums.begin(),nums.end());

    for(int exp=1;mx/exp>0;exp*=10)
        countingSort(nums,exp);
}

```

Excellent question. This code is the heart of Radix Sort, and once you understand Counting Sort, Radix Sort becomes very easy.

We'll dry run it line by line on the example:

```
nums = [170, 45, 75, 90, 802, 24, 2, 66]
```

Overall Idea

Radix Sort calls Counting Sort multiple times.

```
exp = 1 -> sort by Ones digit
exp = 10 -> sort by Tens digit
exp = 100 -> sort by Hundreds digit
```

Notice:

170

Hundreds Tens Ones

1 7 0

First Call

```
countingSort(nums,1);
```

We sort according to the ones digit.

Current array

170
45
75
90
802
24
2
66

Step 1

```
vector<int> cnt(10,0);
```

Initially

[illegible]

Step 2

```
for(int x : nums)
    cnt[(x/exp)%10]++;
```

Since

$exp = 1$

we compute

$(x/1)\%10$

Let's calculate.

$x = 170$

$170/1 = 170$

$170\%10 = 0$

Increase

$cnt[0]++$

Digit	0	1	2	3	4	5	6	7	8	9
Count	1	0	0	0	0	0	0	0	0	0

$x = 45$

$45\%10 = 5$

Digit	0	1	2	3	4	5	6	7	8	9
Count	1	0	0	0	0	1	0	0	0	0

$x = 75$

$75\%10 = 5$

Digit	0	1	2	3	4	5	6	7	8	9
Count	1	0	0	0	0	2	0	0	0	0

$$x = 90$$

$$90 \% 10 = 0$$

Digit	0	1	2	3	4	5	6	7	8	9
Count	2	0	0	0	0	2	0	0	0	0

$$x = 802$$

$$802 \% 10 = 2$$

Digit	0	1	2	3	4	5	6	7	8	9
Count	2	0	1	0	0	2	0	0	0	0

$$x = 24$$

$$24 \% 10 = 4$$

Digit	0	1	2	3	4	5	6	7	8	9
Count	2	0	1	0	1	2	0	0	0	0

$$x = 2$$

$$2 \% 10 = 2$$

Digit	0	1	2	3	4	5	6	7	8	9
Count	2	0	2	0	1	2	0	0	0	0

$$x = 66$$

$$66 \% 10 = 6$$

Final frequency array

Digit	0	1	2	3	4	5	6	7	8	9
Count	2	0	2	0	1	2	1	0	0	0

Step 3

Now comes the most important part.

```
for(int i=1;i<10;i++)  
    cnt[i]+=cnt[i-1];
```

This converts frequency into prefix sum.

Let's calculate.

Initially

2 0 2 0 1 2 1 0 0 0

Now

$\text{cnt}[1] = 0 + 2 = 2$

2 2 2 0 1 2 1 0 0 0

Next

$\text{cnt}[2] = 2 + 2 = 4$

2 2 4 0 1 2 1 0 0 0

Next

$\text{cnt}[3] = 0 + 4 = 4$

2 2 4 4 1 2 1 0 0 0

Next

$\text{cnt}[4] = 1 + 4 = 5$

2 2 4 4 5 2 1 0 0 0

Next

$\text{cnt}[5] = 2 + 5 = 7$

2 2 4 4 5 7 1 0 0 0

Next

`cnt[6]=1+7=8`

2 2 4 4 5 7 8 0 0 0

Remaining

8

8

8

Final

Digit	0	1	2	3	4	5	6	7	8	9
Prefix	2	2	4	4	5	7	8	8	8	8

What does Prefix Mean?

Very important.

`cnt[d]`

means

How many numbers have digit $\leq d$

Example

`cnt[5]=7`

means

7 numbers

have digit ≤ 5

Therefore,
the last position for digit 5 is

$$7 - 1 = 6$$

because indexing starts from 0.

Step 4

Now

```
for(int i=n-1;i>=0;i--)
```

We go from right to left.

This makes Counting Sort stable.

`i=7`

`66`

digit

`6`

`cnt[6]=8`

Place at

`8-1=7`

output

— — — — — **66**

Decrease

`cnt[6]=7`

`i=6`

`2`

digit

`2`

`cnt[2]=4`

Place

`4-1=3`

— — — **2** — — — **66**

Decrease

`cnt[2]=3`

i=5

24

digit

4

cnt[4]=5

Place

5-1=4

_ _ _ 2 24 _ _ 66

i=4

802

digit

2

Now

cnt[2]=3

Place

3-1=2

_ _ 802 2 24 _ _ 66

Notice

802

comes before

2

Exactly like original order.

This is stability.

i=3

90

digit

0

cnt[0]=2

Place

1

_ 90 802 2 24 _ _ 66

i=2

75

digit

5

cnt[5]=7

Place

6

_ 90 802 2 24 _ 75 66

i=1

45

digit

5

Now

cnt[5]=6

Place

5

_ 90 802 2 24 45 75 66

Notice

45

stays before

75

Stable!

i=0

170

digit

0

Now

cnt[0]=1

Place

0

Final

170

90

802

2

24

45

75

66

Why Traverse from Right?

Suppose

45

75

Both have

Digit =5

If we processed from left to right:

45 gets last slot

75 gets previous slot

Result

75
45

Wrong.

If we process from right to left:

75 gets last slot

45 gets previous slot

Result

45
75

Correct.

This preserves the original order of equal digits, making the sort stable, which is essential for Radix Sort to work correctly.

Next Pass (exp = 10)

Now the array is:

170
90
802
2
24
45
75
66

We repeat the same process, but now compute:

```
digit = (x / 10) % 10;
```

That extracts the tens digit:

Number	Tens Digit
170	7
90	9
802	0
2	0
24	2
45	4
75	7
66	6

After Counting Sort (stable), the array becomes:

```
802
2
24
45
66
170
75
90
```

Final Pass (exp = 100)

Now we extract:

```
digit = (x / 100) % 10;
```

Number	Hundreds Digit
802	8
2	0
24	0
45	0
66	0
170	1
75	0
90	0

After the final stable Counting Sort:

2
24
45
66
75
90
170
802

The array is completely sorted.